Corso di Programmazione Web
Esercitazione

Questo esame è a libro aperto, ma tutto il lavoro deve essere svolto individualmente.

Rispondere a ogni domanda in modo chiaro, utilizzando schizzi, tabelle o descrizioni dove necessario.

Non scrivere codice, ma fornire dettagliate progettazioni API.

Scrivere nome, cognome, numero di matricola nel foglio dove si svolge l'esame.

Per risparmiare spazio e tempo, si può usare il seguente formato per descrivere le API:

```
{ idEsempio : int : 1             // assegnato dal server
  campoEsempio : str : "esempio"  // obbligatorio
  altroCampo : int : 1            // opzionale, minimo = 0, massimo =
10 }
```

Sistema di Prenotazione Sale Riunioni

Progettare un'applicazione web che consenta ai dipendenti di un'azienda di prenotare sale riunioni per un determinato giorno e fascia oraria, verificando la disponibilità in tempo reale.

API richieste

- API 1: Creare una nuova prenotazione per una sala
- API 2: Consultare le sale disponibili per una data e fascia oraria
- API 3: Aggiungere un partecipante a una prenotazione esistente

Requisiti dell'applicazione

Ogni sala ha: idSala, nome, piano, capienza, dotazione.

Ogni prenotazione ha: idPrenotazione, idSala, idOrganizzatore, data, oraInizio, oraFine, titolo, dataCreazione.

Gli utenti sono identificati da un idUtente già esistente nel server (non serve API di registrazione).

Specifiche tecniche

- idPrenotazione e dataCreazione sono assegnati dal server.
- Si assuma che l'utente sia assegnato dal server (l'utente ha effettuato il login).
- oraInizio e oraFine sono nel formato HH:MM; oraFine deve essere successiva a oraInizio.
- Non è possibile prenotare la stessa sala per fasce orarie sovrapposte nella stessa data.
- dotazione è una lista di stringhe a scelta libera (es. "proiettore", "lavagna", "videocamera").
- Il numero di partecipanti aggiunto alla prenotazione non può superare la capienza della sala.

Domanda 1 – Progettazione delle API (15 punti)

Progettare le quattro API descritte sopra. Per ciascuna API:

- 1 punto: endpoint e metodo HTTP

- 2 punti: modello di richiesta/risposta completo con tipi di dato e vincoli (inclusi quali campi sono obbligatori/opzionali, quali sono assegnati dal server, ed eventuali massimi e minimi)
- 1 punto: esempio concreto (richiesta e risposta)

Infine, 3 punti in totale sono assegnati in base alla chiarezza e completezza delle informazioni che complementano il servizio (per esempio, menzione specifica di controlli logici/validazioni server-side).

Domanda 2 – Validazione e gestione errori (5 punti)

- 1 punto: descrizione dei seguenti codici di errore (404, 422, 500). Elaborare la risposta – non è sufficiente scrivere il “titolo” del codice di errore.
- 2 punti per ogni esempio: fornire 2 esempi concreti (uno per ogni errore) con messaggio JSON e spiegazione, di richieste che generano i codici di errore 404 e 422. Gli errori devono riferirsi chiaramente alle API appena progettate.

Domanda 3 – Estensione (3 punti)

Attualmente ogni prenotazione riguarda un singolo slot orario non ricorrente. Si vuole aggiungere il supporto a prenotazioni ricorrenti (es. ogni lunedì dalle 10:00 alle 11:00 per le prossime 4 settimane). Descrivere come modificare il modello dei dati e la logica server-side per supportare questo requisito, evidenziando anche come gestire i conflitti su singole occorrenze.

MODELLO PRENOTAZIONE ⟶ {

"IDPRENOTAZIONE" : INT : 1, // ASSEGNATO DAL SERVER, UNIVOCO

"IDSALA" : INT : 3, // SALA PRENOTATA

"IDORGANIZZATORE" : INT : 9, // ASSEGNATO DAL SERVER, UTENTE LOGGATO

"DATA" : STR : "2026-06-04", // OBBLIGATORIO, FORMATO VALIDO yyyy-MM-dd

"ORAINIZIO" : STR : "20:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM

"ORAFINE" : STR : "24:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM ED ESSERE VALIDO SUCCESSIVO AD ORAINIZIO

"TITOLO" : STR : "LAUREA", //OBBLIGATORIO, MAX=50 CARATTERI

"DATACREAZIONE" : STR : "2026060117:33", // ASSEGNATO DAL SERVER, PRESO DALLA DATA ED ORARIO DI PRENOTAZIONE

"PARTECIPANTI" : LIST [INT] : [12, 15] // UTENTI AGGIUNTI, MASSIMO IN BASE ALLA CAPIENZA

}

MODELLO SALA ⟶ {

"IDSALA" : INT : 3, //ASSEGNATO DAL SERVER, UNIVOCO

"NOME" : STR : "ALDO BAGLIO", //OBBLIGATORIO, MAX=100 CARATTERI

"PIANO" : INT : 3, // OBBLIGATORIO

"CAPIENZA" : INT : 20, // OBBLIGATORIO, MIN=1, MAX=100

"DOTAZIONE" : LIST [STR] : ["PROIETTORE", "LAVAGNA"] //OBBLIGATORIO, LISTA ANCHE VUOTA

}

MODELLO UTENTE ⟶ {

"IDUTENTE" : INT : 7, //GIÁ ESISTENTE NEL SERVER

"NOME" : STR : "BRUNO"

}

API 1 - CREAZIONE PRENOTAZIONE PER UNA SALA

POST /SALE/{IDSALA}/PRENOTAZIONI ⟶

SI USA POST PERCHÈ L'UTENTE INVIA DATI AL SERVER PER PRENOTARE UNA SALA

⟵ 201 CREATED

{

"DATA" : STR : "2026-06-04", //OBBLIGATORIO, FORMATO VALIDO yyyy-MM-dd

"ORAINIZIO" : STR : "20:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM

"ORAFINE" : STR : "24:00", // FORMATO VALIDO HH:MM, SUCCESSIVA ORAINIZIO ⟶

"TITOLO" : STR : "LAUREA", // OBBLIGATORIO

"DATACREAZIONE" : STR : "2026060117:33"

}

{

"IDPRENOTAZIONE" : INT : 1, // ASSEGNATO DAL SERVER

"IDSALA" : INT : 3, // ASSEGNATO DAL SERVER

"IDORGANIZZATORE" : STR : "POLDO", //USERNAME

"DATA" : STR : "2026-06-04", //OBBLIGATORIO, FORMATO VALIDO yyyy-MM-dd

"ORAINIZIO" : STR : "20:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM

"ORAFINE" : STR : "24:00", // OBBLIGATORIO, FORMATO VALIDO HH:MM ED ESSERE VALIDO SUCCESSIVO AD ORAINIZIO

"TITOLO" : STR : "LAUREA", //OBBLIGATORIO

"DATACREAZIONE" : STR : "2026060117:33" // ASSEGNATO DAL SERVER, PRESO DALLA DATA ED ORARIO DI PRENOTAZIONE

"PARTECIPANTI" : LIST [INT] : [ ]

}

API 2 - CONSULTAZIONE SALE LIBERE PER DATA ED ORARIO

GET /SALE/DISPONIBILI?DATA=2026-06-04&ORAINIZIO=10:00&ORAFINE=11:00 ⟶

USIAMO GET PERCHÈ VOGLIAMO LEGGERE I DATI, NESSUN BODY

⟵ 200 OK

{

"IDSALA" : INT : 1,

"NOME" : STR : "SALA ADA",

"PIANO" : INT : 2,

"CAPIENZA" : INT : 12,

"DOTAZIONE" : LIST [STR] : ["PROIETTORE,..."]

}

API 3 - AGGIUNGERE UN PARTECIPANTE AD UNA PRENOTAZIONE ESISTENTE

POST /PRENOTAZIONI/{IDPRENOTAZIONE}/PARTECIPANTI →

SI USA POST PERCHÈ L'UTENTE STA MODIFICANDO UNA RISORSA NEL SISTEMA.

← 200 OK

```
{
  "IDUTENTE": INT: 12, //OBBLIGATORIO, UTENTE GIÀ ESISTENTE NEL SERVER
}
```

⟶

```
{
  "IDPRENOTAZIONE": INT: 10,
  "IDSALA": INT: 1,
  "IDORGANIZZATORE": STR: "POLDO",
  "DATA": STR: "2026-06-04",
  "ORAINIZIO": STR: "20:00",
  "ORAFINE": STR: "24:00",
  "TITOLO": STR: "LAUREA",
  "DATACREAZIONE": STR: "2026060117:13:33"
  "PARTECIPANTI": LIST[INT]: [12]
}
```

- 404 NOT FOUND ⟶ INDICA CHE LA RISORSA RICHIESTA NON ESISTE. SI USA QUANDO IL CLIENT CHIEDE UNA SALA, UNA PRENOTAZIONE O UN ENDPOINT INESISTENTE.

POST /SALE/99/PRENOTAZIONI →

← 404 NOT FOUND

```
{
  "DATA": "2026-06-04",
  "ORAINIZIO": "10:00",
  "ORAFINE": "11:00",
  "TITOLO": "LAUREA"
}
```

⟶

```
{
  "ERROR": "SALA CON IDSALA 99 NON TROVATA"
}
```

- 422 UNPROCESSABLE ENTITY ⟶ INDICA CHE LA RICHIESTA È SINTATTICAMENTE CORRETTA, MA CONTIENE DATI CHE VIOLANO VINCOLI LOGICI O DI VALIDAZIONE. SI USA PER ORARI NON VALIDI, FASCE ORARIE SOVRAPPOSTE, CAPIENZA SUPERATA, UTENTE GIÀ PARTECIPANTE O FORMATO DATA/ORARIO ERRATO

POST /SALE/1/PRENOTAZIONI →

← 422 UNPROCESSABLE ENTITY

```
{
  "DATA": "2026-06-04",
  "ORAINIZIO": "13:00",
  "ORAFINE": "10:00",
  "TITOLO": "LAUREA"
}
```

⟶

```
{
  "ERROR": "ORAFINE DEVE ESSERE SUCCESSIVA AD ORAINIZIO"
}
```

- 500 INTERNAL SERVER ERROR ⟶ INDICA UN ERRORE INTERNO LATO SERVER. LA RICHIESTA PUÓ ANCHE ESSERE CORRETTA, MA IL SERVER NON RIESCE A COMPLETARLA. SI USA SE IL DATABASE NON È DISPONIBILE, SE IL SALVATAGGIO DELLA PRENOTAZIONE FALLISCE O SE SI VERIFICA UN'ECCEZIONE NON GESTITA

POST /SALE/1/PRENOTAZIONI →

← 500 INTERNAL SERVER ERROR

```
{
  "DATA": "2026-06-04",
  "ORAINIZIO": "12:00",
  "ORAFINE": "13:00",
  "TITOLO": "LAUREA"
}
```

⟶

```
{
  "ERROR": "ERRORE INTERNO DEL SERVER. RIPROVA PIÚ TARDI"
}
```

PRENOTAZIONI RICORRENTI ⟶ PER SUPPORTARE PRENOTAZIONI RICCORENTI, IL MODELLO DELLA PRENOTAZIONE PUÓ ESSERE ESTESO AGGIUNGENDO CAMPI COME RICCORENTE, FREQUENZA, NUMEROCCORENZE O DATAFINE ED IDSERIE, CHE COLLEGA TUTTE LE PRENOTAZIONI APPARTENENTI ALLA STESSA SERIE. PER ESEMPIO, UNA PRENOTAZIONE OGNI LUNEDÍ DALLE 10:00 ALLE 11:00 PER QUATTRO SETTIMANE AVREBBE QUATTRO OCCORRENZE COLLEGATE DALLO STESSO IDSERIE. LATO SERVER, QUANDO RICEVE UNA RICHIESTA RICORRENTE, IL SISTEMA NON DEVE SALVARE UNA SOLA PRENOTAZIONE GENERICA, MA DEVE GENERARE LE SINGOLE OCCORRENZE NELLE DATE PREVISTE. PER OGNI OCCORRENZA DEVE CONTROLLARE DISPONIBILITÀ DELLA SALA, FORMATO DEGLI ORARI, SOVRAPPOSIZIONI E CAPIENZA. SE ANCHE UNA SOLA OCCORRENZA ENTRA IN CONFLITTO CON UNA PRENOTAZIONE GIÁ ESISTENTE, IL SERVER PUÓ RIFIUTARE TUTTA LA SERIE CON ERRORE 422